SECTION 7: SECURITY AND ZERO TRUST — DEFENDING THE KINGDOM ⏱️ 9 min • MODULE 26

Part 26: Authentication & Authorization at Scale: OAuth2, OIDC, and RBAC vs. ABAC

Decoupling identity verification from fine-grained permissions without building header monsters.

🔥 **WAR STORY FROM THE TRENCHES**

A healthcare enterprise built a telemedicine platform. The architecture team decided to encode all user permissions directly into a JSON Web Token (JWT) payload so services wouldn't have to query the database. For doctors affiliated with hospital networks, their permissions included 400 clinic IDs, 80 hospital wings, and 50 specialty procedure codes. By month six, the JWT payload ballooned to 28 kilobytes! When doctors submitted consultation requests, NGINX reverse proxies and API gateways threw `HTTP 431 Request Header Fields Too Large` errors, blocking doctors from accessing patient charts during live emergencies. Worse, when a doctor was suspended for malpractice, their JWT remained valid for 24 hours because the stateless token couldn't be easily revoked.

The Paradigm Shift: From Coder to Architect Mindset

Authentication (AuthN: **Who are you?**) and Authorization (AuthZ: **What are you allowed to do?**) are fundamentally different architectural problems. OAuth 2.0 and OpenID Connect (OIDC) have solved AuthN and delegated access. But AuthZ at scale cannot be solved by cramming static roles into giant JWT tokens. Modern architectures separate coarse-grained authentication (identity tokens) from centralized, dynamic authorization engines using **Role-Based Access Control (RBAC)** or **Attribute-Based Access Control (ABAC)**.

Core Architectural Principles & Mental Models

Auth Protocols Demystified

- **OpenID Connect (OIDC):** Built on top of OAuth 2.0. Issues an `id_token` (JWT) to prove user identity and profile information.
- **OAuth 2.0:** Authorization framework. Issues an `access_token` allowing a client application to access resources on behalf of the user.
- **RBAC (Role-Based Access Control):** Permissions are assigned to roles (Admin, Editor, Viewer). Simple, intuitive, but suffers from 'Role Explosion' as enterprise rules grow.
- **ABAC / ReBAC (Attribute / Relationship-Based Access Control):** Policy decisions are based on attributes: `Can User X edit Document Y IF User.Department == Document.Department AND Time \< 18:00?` (e.g., Open Policy Agent - OPA, Google Zanzibar, Oso).

Token Revocation Strategies

Keep JWT lifetimes short (5 to 15 minutes). Use long-lived opaque refresh tokens stored in a distributed Redis/database revocation blacklist for instant revocation.

Common Anti-Patterns & Traps to Avoid

- **The 50-Kilobyte JWT Monster:** Shoving every permission, address, and profile detail into the JWT header instead of using lightweight subject IDs.
- **Hardcoded Authorization in Controllers:** Writing `if (user.role == 'manager' && user.region == 'EU')` deep inside business service classes, scattering authorization logic everywhere.
- **Roll-Your-Own Crypto & Auth:** Attempting to write your own password hashing algorithm or session token generator instead of using proven libraries (Argon2, bcrypt, standard OAuth providers).

> 🎯 **THE ARCHITECT'S PLAYBOOK: ACTION STEPS THIS WEEK**
>
> - **Step 1:** Adopt OIDC/OAuth 2.0 using a dedicated identity provider (Auth0, Okta, Keycloak, or AWS Cognito).
> - **Step 2:** Centralize authorization policies using a policy engine like Open Policy Agent (OPA) or Oso rather than hardcoding if/else checks.
> - **Step 3:** Set access token expiration to 15 minutes with automated sliding refresh token rotation.

Architectural Takeaways & Cheatsheet

- AuthN tells you who is knocking at the door; AuthZ determines which rooms they are permitted to enter.
- Keep stateless tokens lightweight; fetch fine-grained permissions dynamically or through a specialized policy sidecar.
- Never invent your own cryptographic authentication system; use audited industry standards.

> 🎙️ **UDEMY & NOTEBOOKLM DISCUSSION PROMPT**
>
> **Course Reflection & Hands-on Assignment:** How does your system handle instant permission revocation? If an employee is terminated right now, does their active session terminate immediately or do their tokens work until they expire?